Planning

Introduction

- Planning is required to reach a particular destination (Goal).
- It is necessary to find the best route but the tasks to be done at **a particular time and why** they are done also very important.
- Planning is considered the **logical side of taking action.**
- Planning is about deciding the tasks to be performed by the AI system and the system's functioning under **domain-independent** conditions.
- We require **domain description**, **task specification**, and **goal description** for any planning system.
- *A plan is considered a sequence of actions, and each action has its preconditions that must be satisfied before it can act and some effects that can be positive or negative.*

Forward State Space Planning (FSSP)

- Forward State Space Planning (FSSP) and Backward State Space Planning (BSSP) at the basic level.
- FSSP behaves in the same way as forwarding state-space search.
- It says that given an initial state S in any domain, we perform some necessary actions and obtain a new state S', called a progression.
- It continues until we reach the target position.
- **Disadvantage**: Large branching factor
- **Advantage**: The algorithm is Sound

BSSP

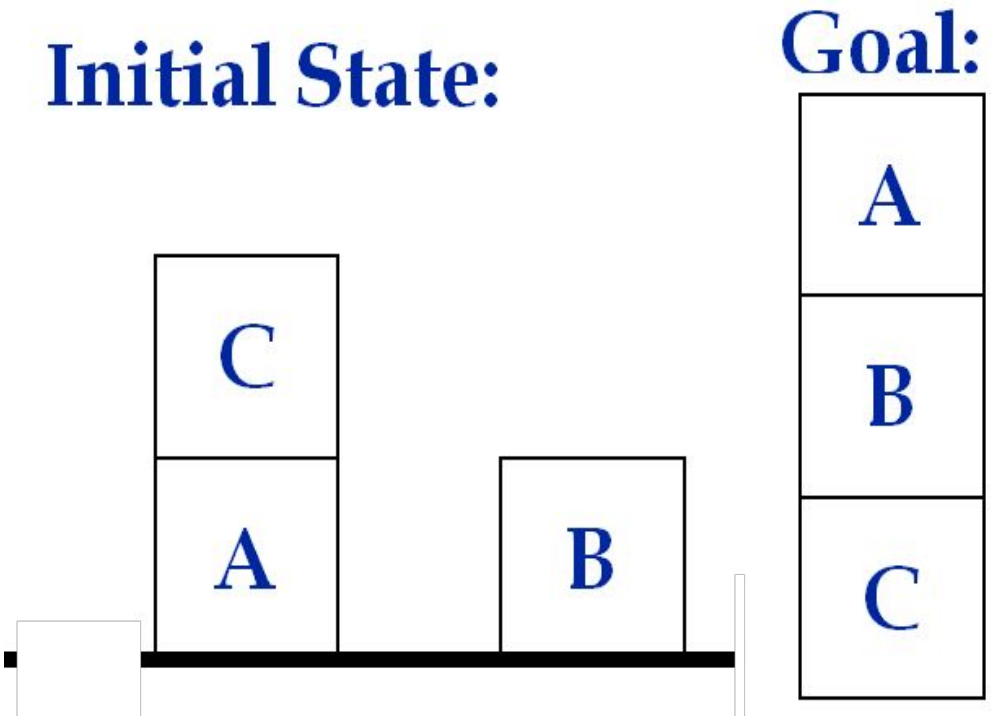
- Move from the target state *g* to the sub-goal of *g*, tracing the previous action to achieve that goal.
- This process is called regression.
- These sub-goals should also be checked for **consistency** i.e. the action should be relevant.
- **Disadvantages**: not sound algorithm (sometimes inconsistency can be found)
- **Advantage**: Small branching factor (much smaller than FSSP)
- So for an efficient planning system, we need to combine the features of FSSP and BSSP.

Components of the planning system

- Choose the best rule to apply the next rule based on the best available guess.
- Apply the chosen rule to calculate the new problem condition.
- Find out when a solution has been found.
- Detect dead ends so they can be discarded and direct system effort in more useful directions.
- Find out when a near-perfect solution is found.

Sussman Anomaly

- The Sussman Anomaly shows the limitations **of non-interleaved planning methods**.

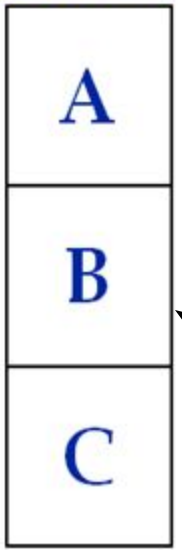


- The anomaly will show that naively pursuing one subgoal X after you satisfy the other subgoal Y, may not work because steps required to accomplish X may undo steps of subgoal Y.

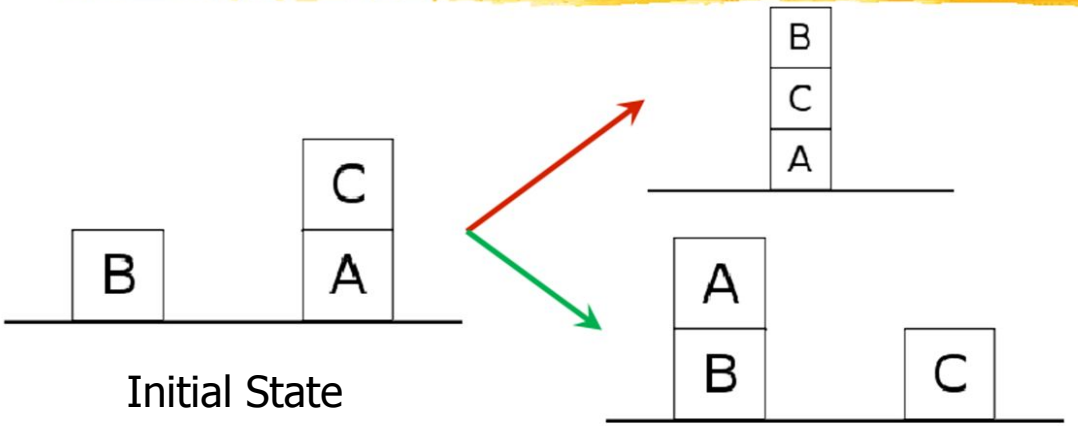
Initial State: (on-table A), (on C A), (on-table B), (clear B), (clear C)
Goal: (on A B) (on B C)

Anomaly Illustrates the Need for Interleaved Plans

Goal:



Sussman Anomaly



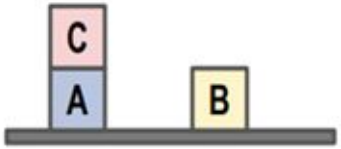
- Final state requires On(A,B) and On(B, C)
- Red arrow focus on subgoal: On(B,C) -- Now trying to put A on top of B cannot be done without undoing On(B, C)
- Bottom diagram tries to focus on subgoal: On(A, B) first; but now trying to put B on top of C would cause On(A,B) to be undone!

Partial Solutions

ACTIONS:

Move(X, Y)
Precond: Clear(X), Clear(Y)
Effect: On(X, Y)

Move(X, Table)
Precond: Clear(X)
Effect: On(X, Table)



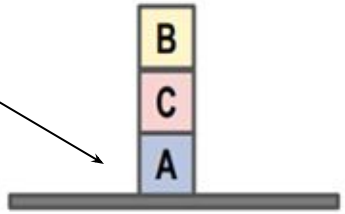
On(C, A), On(A, Table), On(B, Table), Clear(C), Clear(B)

Clear(C), Clear(B)

Move(B, C)

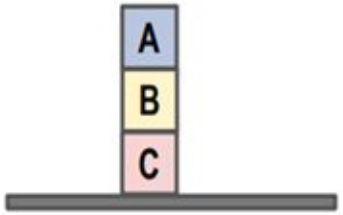
We use Move(B, C) to achieve the sub-goal, On(B, C).

But if we apply this move at the beginning, we get:



Which is not what we want !!

On(A, B), On(B, C)



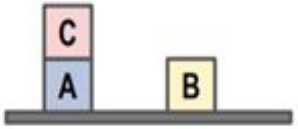
Goal state

Partial Solutions

ACTIONS:

Move(X, Y)
Precond: Clear(X), Clear(Y)
Effect: On(X, Y)

Move(X, Table)
Precond: Clear(X)
Effect: On(X, Table)



On(C, A), On(A, Table), On(B, Table), Clear(C), Clear(B)

Clear(C)

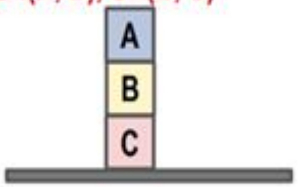
Move(C, Table)

Clear(A), On(C, Table)

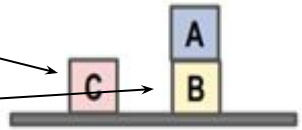
Clear(A), Clear(B)

Move(A, B)

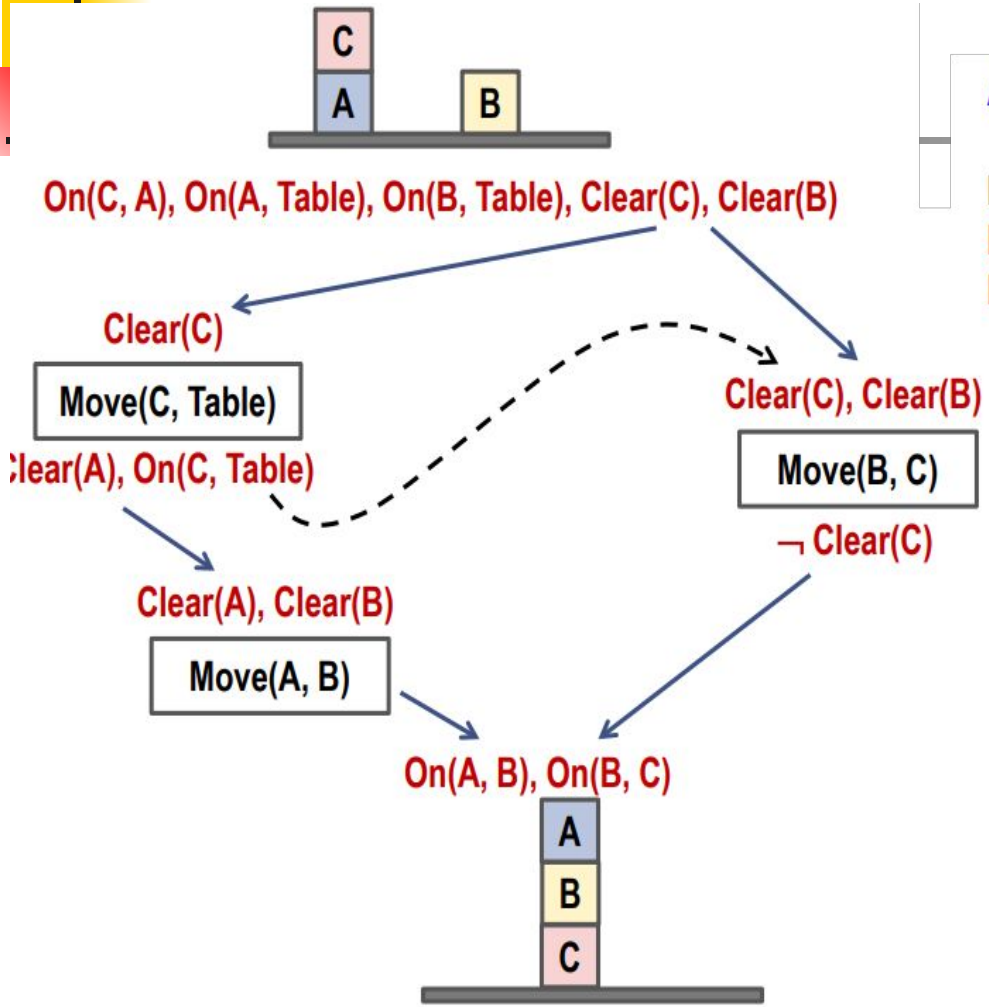
On(A, B), On(B, C)



The sub-goal On(A, B) is achieved by moving C to the table and then moving A to top to B. But this gives us:



But this too is not what we want !!



ACTIONS:

Move(X, Y)
Precond: Clear(X), Clear(Y)
Effect: On(X, Y)

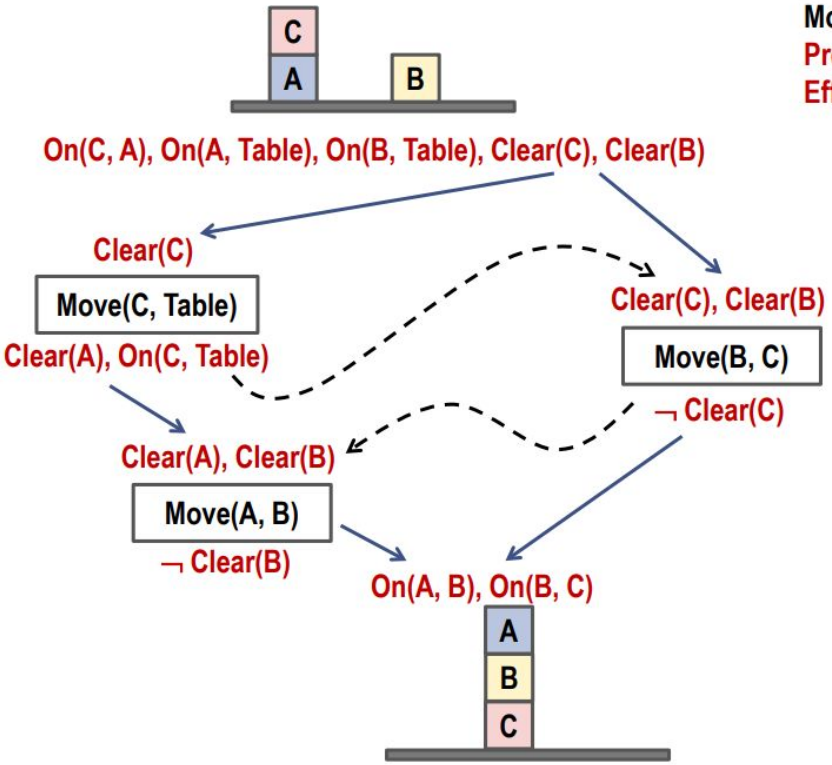
Move(X, Table)
Precond: Clear(X)
Effect: On(X, Table

Move(B, C) removes the Clear(C) predicate which is essential for Move(C, Table). Hence Move(C, Table) must precede Move(B, C).

Can Move(B, C) and Move(A, B) be executed in any order?

Can Move(B, C) and Move(A, B) be executed in any order?

Ordering Partial Solutions



ACTIONS:

Move(X, Y)
Precond: Clear(X), Clear(Y)
Effect: On(X, Y)

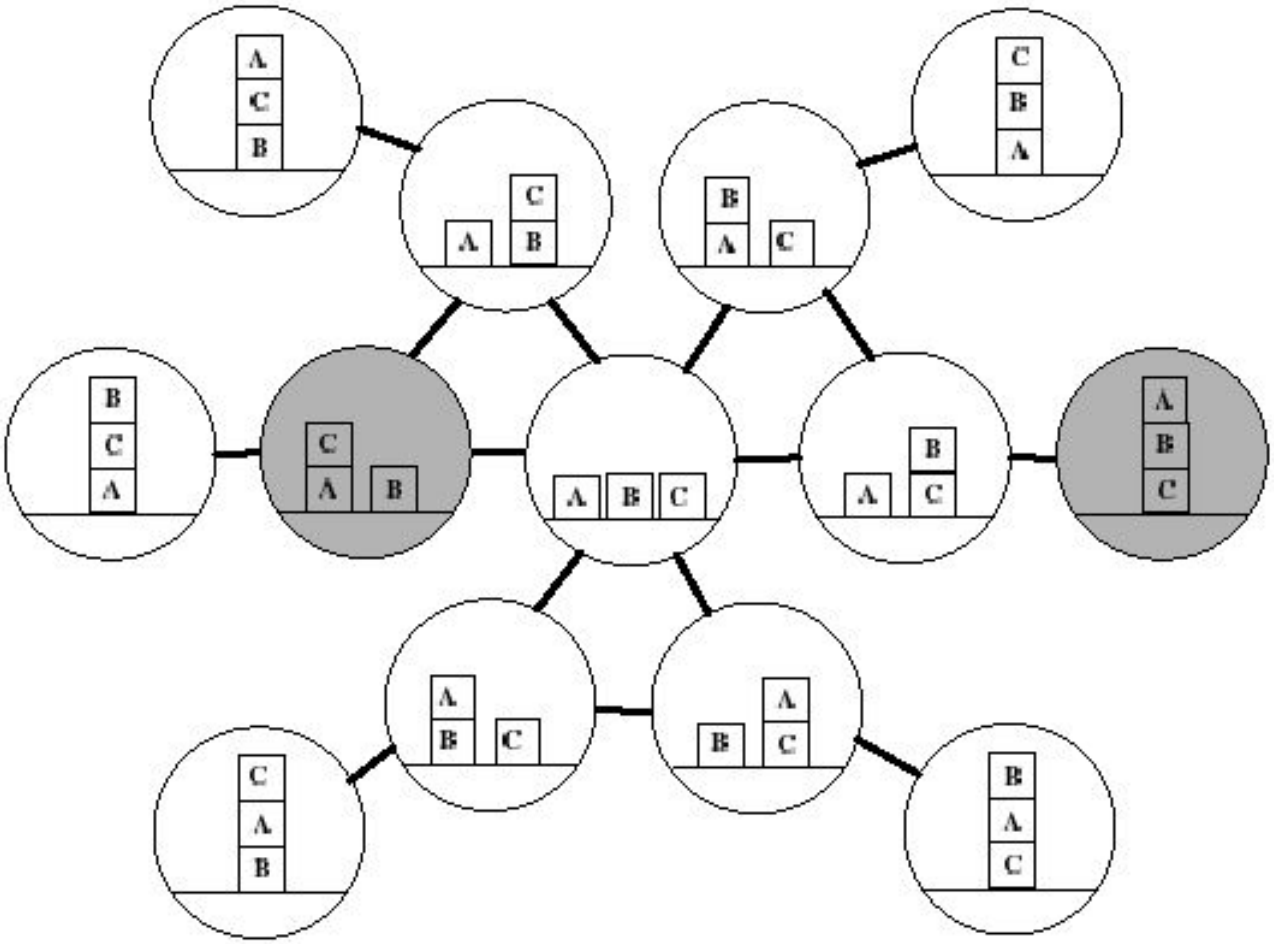
Move(X, Table)
Precond: Clear(X)
Effect: On(X, Table)

Move(A, B) removes the Clear(B) predicate which is essential for Move(B, C). Hence Move(B, C) must precede Move(A, B).

Therefore the only total order is:

1. Move(C, Table)
2. Move(B, C)
3. Move(A, B)

Search Space: Blocks World



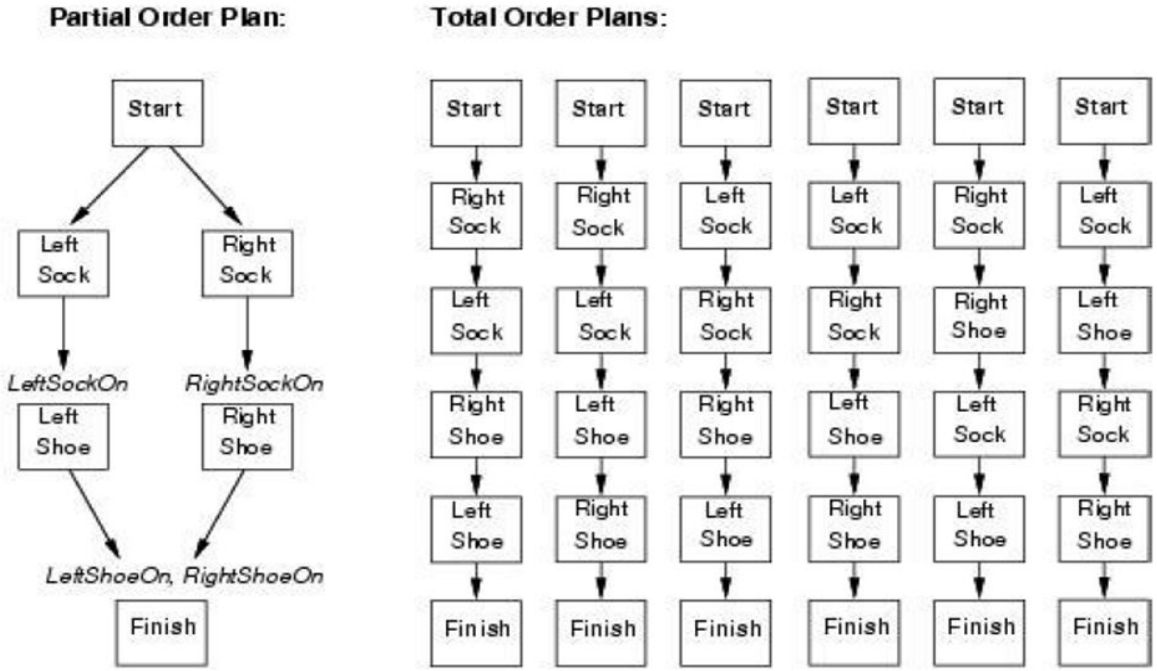
Partial-order planning

- Progression and regression planning are *totally ordered plan search* forms.
  - They cannot take advantage of problem decomposition.
    - Decisions must be made on how to sequence actions on all the subproblems
- Least commitment strategy:
  - Delay choice during search

- **Basic Idea: Make choices only that are relevant to solving the current part of the problem**

Partial-order planning

- Any planning algorithm that can place two actions into a plan without commitment about which comes first is a Partial Order Plan



- A partial order plan is executed by repeatedly choosing *any* of the possible next actions.
  - This flexibility is a benefit in non-cooperative environments.

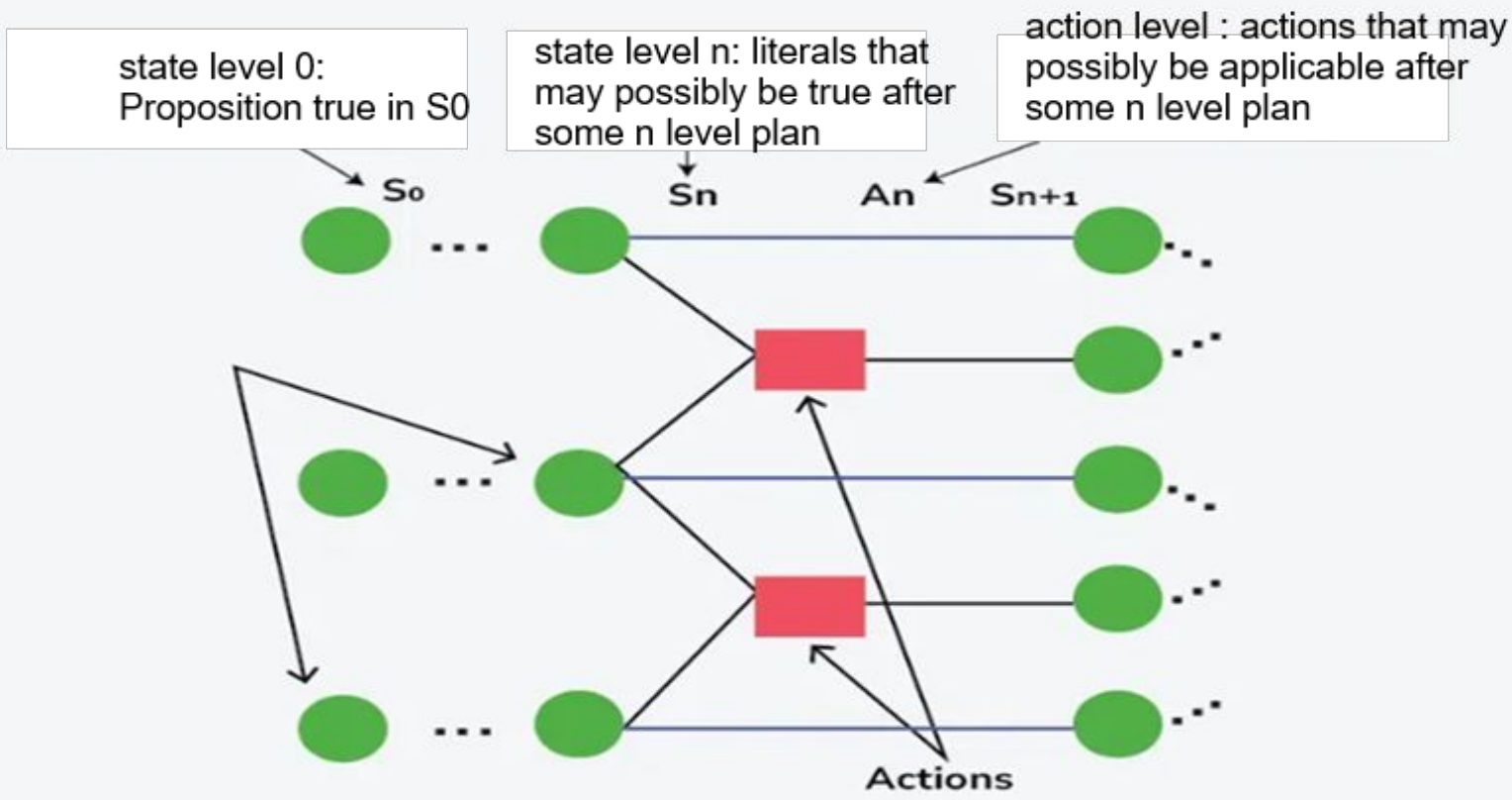
Target stack plan

- It is one of the most important planning algorithms used by STRIPS. • STanford Research Institute Problem Solver
- Stacks are used in algorithms to capture the action and complete the target.
- A knowledge base is used to hold the current situation and actions.
- A target stack is similar to a node in a search tree, where branches are created with a choice of action.
- Construct and analyze the compact structure called **graph planning.**

What is a Planning Graph?

- A Planning Graph is a data structure primarily used in automated planning and progression through a series of levels that describe states of the world and the actions that evolve to achieve the goal.
- They provide efficient reasoning about which actions can lead to the goal state and which combinations are infeasible due to conflicts.
- It helps identify mutual exclusions (mutex) between actions or propositions that cannot occur together.
- The GraphPlan algorithm uses the planning graph to extract valid plans efficiently.
- Planning graphs are especially useful in STRIPS-like domains where the state space is large but actions have structured preconditions and effects.

Planning Graph



Components of Planning Graphs

- **Levels:** A Planning graph has two alternating types of levels: **action levels and state levels**. The first level is always a state level, representing the initial state of the planning problem.
- **State Levels** consist of nodes representing logical propositions or facts about the world. Each successive state level contains all the propositions of the previous level plus any that can be derived by the actions of the intervening action levels.
- **Action Levels** contain nodes representing actions. An action node connects to a state level if the state contains all the preconditions necessary for that action. Actions in turn can create new state conditions, influencing the subsequent state level.
- **Edges:** one connecting state nodes to action nodes (indicating that the state meets the preconditions for the action) and another connecting action nodes to state nodes (indicating the effects of the action).

Structure of Planning Graph

- A Planning Graph alternates between two types of levels:

1. State Levels(S_i): Each state level represents a set of propositions (facts) that could hold true at a particular step. It contains all the propositions from the previous state plus any new ones generated by actions.

$$S_i = \{p \mid p \text{ is true at level } i\}$$

2. Action Levels(A_i): Each action level contains all actions whose preconditions are satisfied by the propositions in the preceding state level.

$$A_i = \{a \mid \text{Preconditions}(a) \subseteq S_i\}$$

Mutual Exclusion (Mutex) Relationships: At each level, certain pairs of actions or states might be mutually exclusive, meaning they cannot coexist or occur together due to conflicting conditions or effects.

These mutex relationships are critical for reducing the complexity of the planning problem by limiting the combinations of actions and states that need to be considered.

A Planning Graph begins with an initial state and grows iteratively through alternating layers of state levels and action levels. At each iteration, the graph models how the world evolves as actions are applied, until either a valid plan is found or no new states can be generated.

- **Step 1: Initialization:**
- Begin with the initial state level S_0, which contains all propositions (facts) known to be true at the start of the problem.
- This level forms the foundation for constructing possible future actions and states.
- **Step 2: Expansion Phase:**
- From the current state level Si identify all actions a whose preconditions are satisfied by S_i.
- These actions are added to the next action level A_i.
- $A_i = \{a \mid \text{Pre conditions}(a) \subseteq S_i\}$
- This step enumerates every possible action that can occur at time step *i*, given the current world state.

- **Step 3: Resulting State Level Generation:**
- For each action in A_i, add its positive effects (Add list) and remove its negative effects (Delete list) to create the next state level S_{i+1}.
- $S_{i+1} = S_i + \text{Add}(A_i) - \text{Delete}(A_i)$
- This level represents all possible states that can result from executing the available actions at level A_i.
- **Step 4: Checking for Goal Satisfaction:**
- After generating S_{i+1}, check if all goal propositions appear in it and none of them are mutually exclusive (non-mutex).
- If this condition is met, a valid plan can potentially be extracted by backward search.
- Goal reachable if $G \subseteq S_{i+1} \wedge \neg\text{Mutex}(G)$

- **Step 5: Termination Condition**
- If a valid plan is found, the algorithm stops and extracts it.
- If no new actions or states appear (the graph levels off) and the goal is still unreachable, planning fails.
- $S_{i+1} = S_i \Rightarrow$ Graph leveled off (no new information)

Plan Generation: Search space of world states

Planning as a (graph) search problem

- Nodes: world states
- Arcs: actions
- Solution: path from the initial state to one state that satisfies the goal
  - Initial state is fully specified
  - There are many goal states

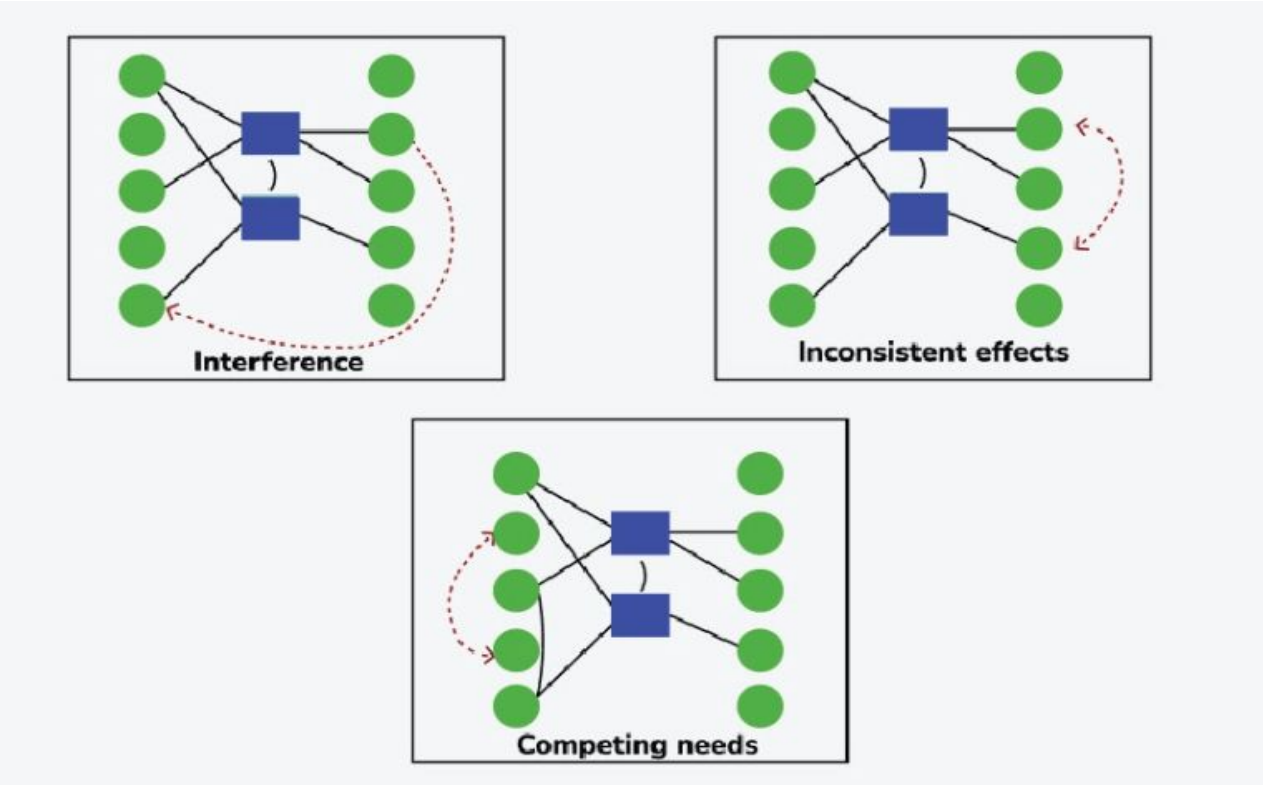
Planning Graphs

- Consists of a sequence of levels that correspond to time steps in the plan
- Each level contains a set of actions and a set of literals that *could* be true at that time step depending on the actions taken in previous time steps
- For every +ve and –ve literal C, we add a *persistence action* with precondition C and effect C

Mutex Conditions Between Actions

Inconsistent Effects: One action negates the effect of another.

- **Interference**: One action deletes a precondition or creates an add-effect of another.
- **Competing Needs**: Precondition of action *a* and precondition of action *b* cannot be true simultaneously.



Two literals are mutually exclusive if one is the negation of the other.

Cake Example

- They work only for propositional problems.
- Example:

Init(Have(Cake))

Goal(Have(Cake) $\wedge$ Eaten(Cake))

Action(Eat(Cake), PRECOND: Have(Cake)

EFFECT: $\neg$Have(Cake) $\wedge$ Eaten(Cake))

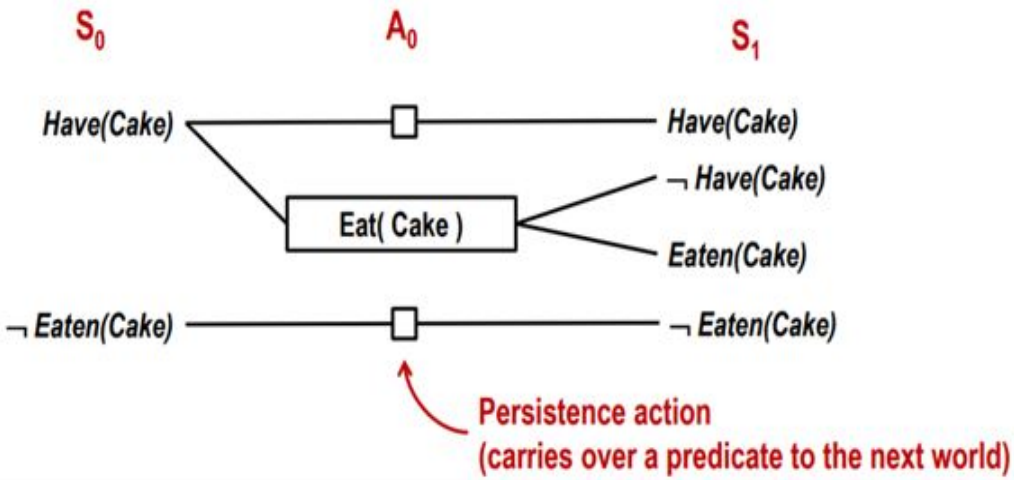
Action(Bake(Cake), PRECOND: $\neg$ Have(Cake)

EFFECT: Have(Cake))

Planning Graph

Op(ACTION: Eat(Cake),
PRECOND: Have(Cake),
EFFECT: Eaten(Cake) ∧ ¬Have(Cake))

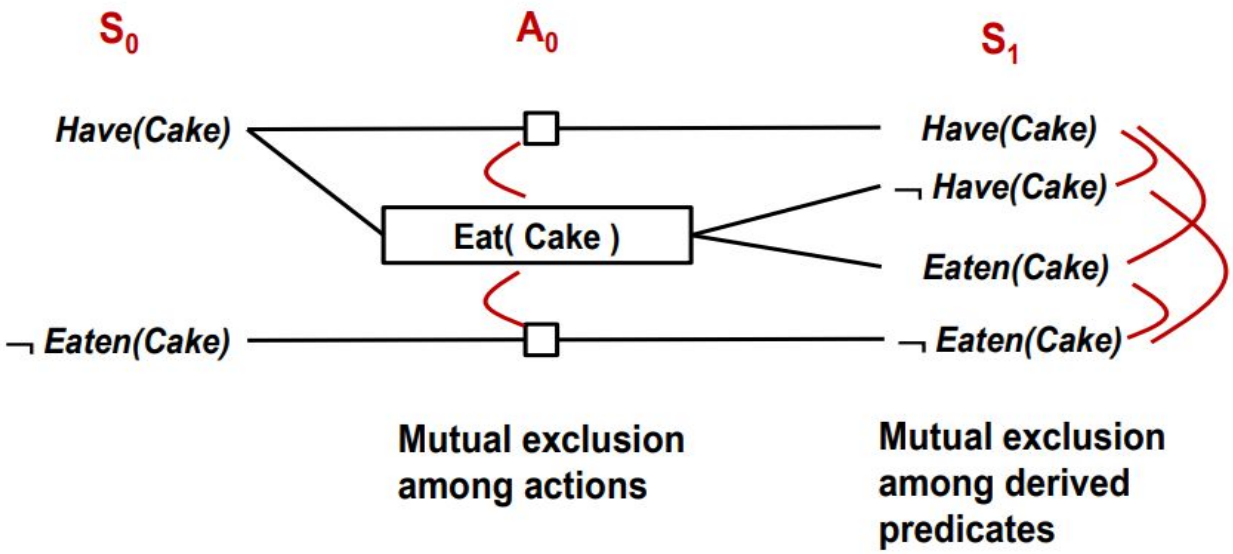
Start: Have(Cake)
Finish: Have(Cake) ∧ Eaten(Cake)



The mutex links define the set of states, revealing which combinations of literals are not possible together.

Mutex Links in a Planning Graph

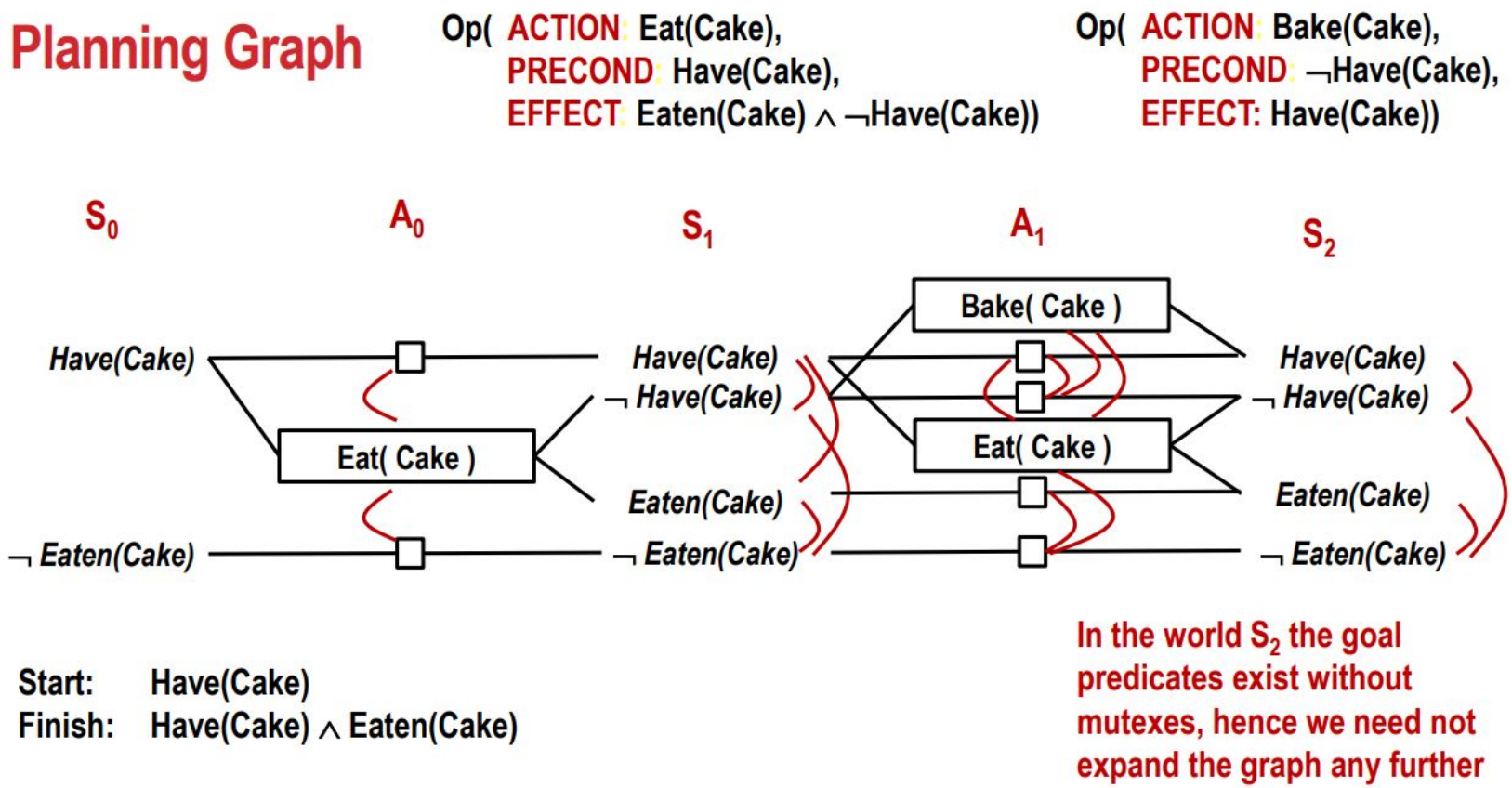
If Eat(Cake) is not performed, ¬**Have(Cake)** and **Eaten(Cake)** cannot be true together.



if we eat the cake, we cannot have the cake at the same time.

Thus, **Have(Cake)** and **Eaten(Cake)** would be mutually exclusive.

Second Action Level (A1) and State Level (S2)



Bake(Cake): The precondition for this action is **¬Have(Cake)**. If **¬Have(Cake)** is true in S1, applying Bake(Cake) results in **Have(Cake)** in S2.
This action does not affect the **Eaten(Cake)** or **¬Eaten(Cake)** states.
Eat(Cake): The precondition for this action is **Have(Cake)**. If **Have(Cake)** is true in S1, applying Eat(Cake) results in **¬Have(Cake)** and **Eaten(Cake)** in S2.

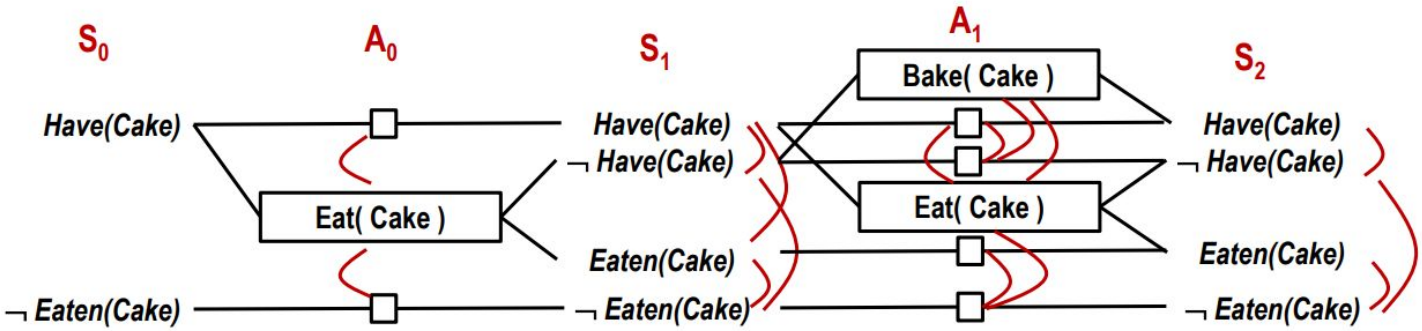
Mutex Actions

- **Mutex relation exists between two actions if:**
  - **Inconsistent effects – one action negates an effect of the other**

    Eat(Cake) causes ¬ *Have(Cake)* and Bake(Cake) causes *Have(Cake)*
  - **Interference – one of the effects of one action is the negation of a precondition of the other**

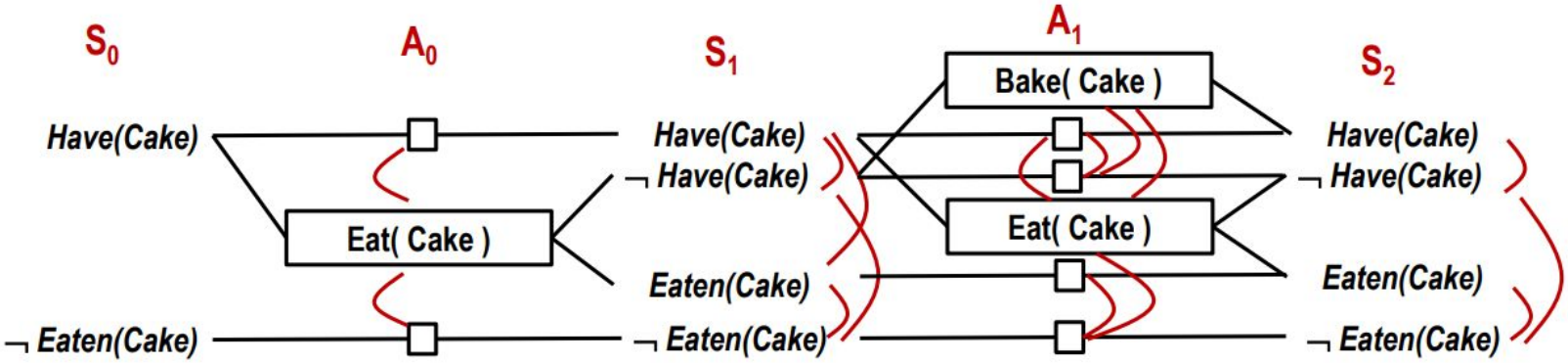
    Eat(Cake) causes ¬ *Have(Cake)* and the persistence of *Have(Cake)* needs *Have(Cake)*
  - **Competing needs – one of the preconditions of one action is mutually exclusive with a precondition of the other**

    Bake(Cake) needs ¬ *Have(Cake)* and Eat(Cake) needs *Have(Cake)*



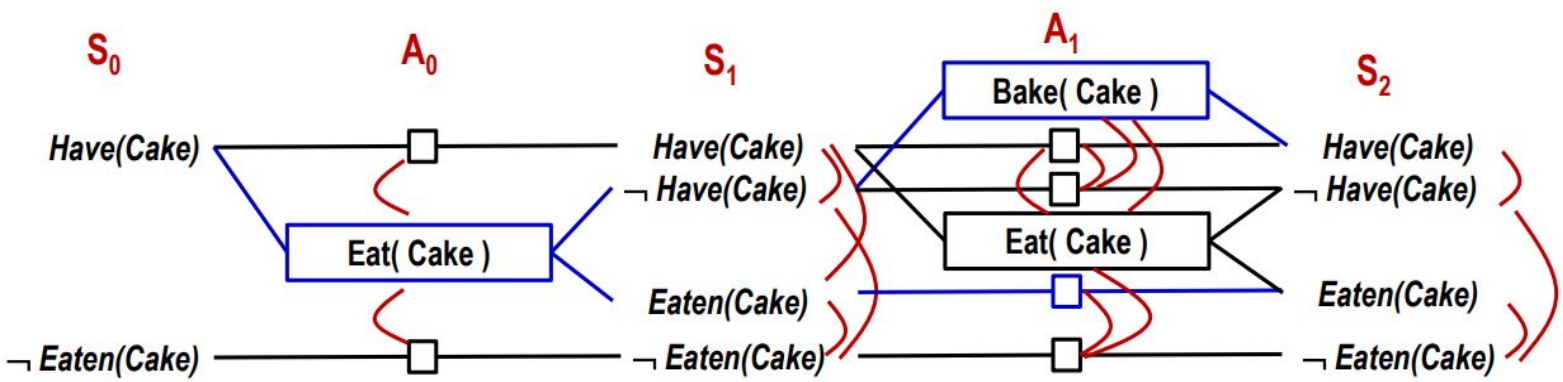
Mutex Literals

- **Mutex relation exists between two literals if:**
  - One is the negation of the other, or
  - Each possible pair of actions that could achieve the two literals is mutually exclusive (inconsistent support)



```
Function GraphPLAN( problem )
    // returns solution or failure
    graph ← Initial-Planning-Graph( problem )
    goals ← Goals[ problem ]
    do
        if goals are all non-mutex in last level of graph then do
            solution ← Extract-Solution( graph )
            if solution ≠ failure then return solution
            else if No-Solution-Possible (graph )
                then return failure
        graph ← Expand-Graph( graph, problem )
```

```
Function GraphPLAN( problem )
    // returns solution or failure
    graph ← Initial-Planning-Graph( problem )
    goals ← Goals[ problem ]
    do
        if goals are all non-mutex in last level of graph then do
            solution ← Extract-Solution( graph )
            if solution ≠ failure then return solution
            else if No-Solution-Possible (graph )
                then return failure
        graph ← Expand-Graph( graph, problem )
```

Finding the plan

- Once a world is found having all goal predicates without mutexes, the plan can be extracted by solving a constraint satisfaction problem (CSP) for resolving the mutexes
- Creating the planning graph can be done in polynomial time, but planning is known to be a PSPACE-complete problem. The hardness is in the CSP.
- The plan is shown in blue below